Compiler Project Part 3

By: Meah Tahmeed Ahmed

Adopted from slides developed by Zach Patterson

How to approach

- Create a symbol table
- Perform type analysis

TypeCheckingTest.java

- Perform type checking
- Print out errors if there is any
- [Feel free to work on this file from the repository.]

```
Lexer lexer = new Lexer(reader);
parser parser = new parser(lexer);
Program program = null;
try {
  program = (Program) parser.parse().value;
}
catch (Exception e) {
  e.printStackTrace();
}
if (program != null) {
  try {
    String typeCatch = program.typeCheck();
    System.err.println("All good!");
  } catch (UTDLangException u) {
    System.err.println(u.toString());
  }
}
```

UTDLangException.java

- Extends Exception.java
- Customize error message

```
class UTDLangException extends Exception
{
   String error;
   public UTDLangException(String s)
   {
       error = s;
   }
   public String toString()
   {
     return error;
   }
}
```

Symbol Table

- What it must do
  - Be shared across all nodes in AST
  - Match variables to their type
  - Deal with different levels of scope
    - Global
    - Method
    - Scope statement 1 (Compound statement, If statement, While statement)
    - Scope statement 2
    - …
    - Scope statement N

Symbol Table

- Share across all nodes
  Static data field in Token class
  https://github.com/pattersonzUTD/UTDLang–/blob/main/Token.java

```
abstract class Token {
  protected static SymbolTable symbolTable;
  protected String getTabs(int t)
 {
   String tabs = "";
   for (int i = 0; i < t; i++)
     tabs = tabs + "\t";
   return tabs;
 }
……
```

Symbol Table

- Match variables ID to Type
  - Approaches
    - HashMap<String, type>
    - List<Pair<String, type>>
    - Either works fine
  - In the example, I use List<Pair<String,String>> where I pair the ID with the type represented as a string

```
class SymbolTable
{
  ArrayList<ArrayList<Pair<String,String>>> table;
  public SymbolTable()
  {
    table = new ArrayList<ArrayList<Pair<String,String>>>();
    table.add(new ArrayList<Pair<String,String>>());
  }
```

Symbol Table

- Dealing with new scopes
- Start and end scope methods

```
public void startScope()
{
  table.add(new ArrayList<Pair<String,String>>());
}

public void endScope()
{
  table.remove(table.size()-1);
}
```

Symbol Table

- Adding variables into a scope

```
public void addVar(String id, String t) throws UTDLangException
{
  for (Pair<String,String> p : table.get(table.size()-1))
    {
      if (p.getKey().equals(id))
        throw new UTDLangException("Error: tried to redeclare variable " + id);
    }
  table.get(table.size()-1).add(new Pair<String,String>(id,t));

  return;
}
```

Symbol Table

- Adding routines into a scope

```
public void addRoutine(String id, InOutList params) throws UTDLangException
{
  String pType = params.getType();

  for (Pair<String,String> p : table.get(table.size()-1))
    {
      if (p.getKey().equals(id))
        throw new UTDLangException("Error: tried to redeclare routine " + id);
    }
  table.get(table.size()-1).add(new Pair<String,String>(id,pType));

  return;
}
```

Symbol Table

- Getting variables from a scope

```
public String get(String s) throws UTDLangException
{
  for (int i = table.size()-1; i >= 0; --i)
    for (Pair<String,String> p : table.get(i))
    {
      if (p.getKey().equals(s))
        return p.getValue();
    }
  throw new UTDLangException("Error: variable not declared " + s);
}
```

Type

- In the example, my types are just string, number, and flag, and I have no coercion.
- Not only do you have more types, but these may also be **‘final’** or an **array**.
- You also have various levels of **coercion** which if done strictly through switch statements or ‘if else’ statements will be massive and cumbersome.
- While I use a String, I’d recommend you **create a Type object**.

Type

- While I use a String, I'd recommend you **create a Type object**.

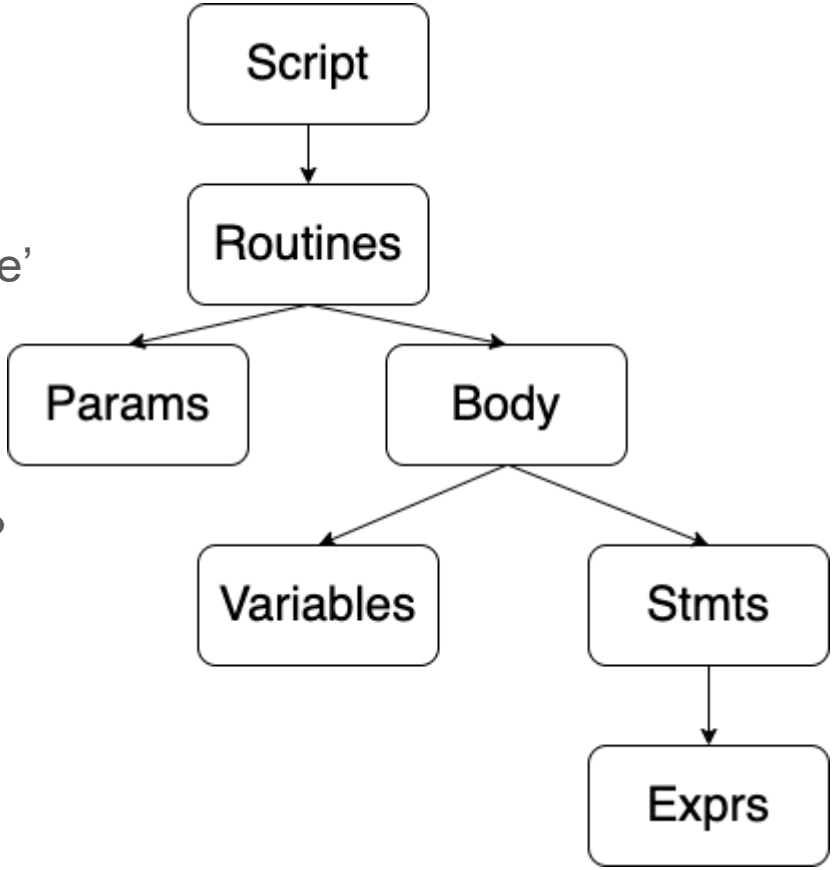
```
class FullType
{
    private Type type;
    private boolean isArray;
    private boolean isFinal;

    FullType(Type t, boolean a, boolean f)
    {
        type = t;
        isArray = a;
        isFinal = f;
    }
```

```
public boolean equals(FullType f)
    {
        return isArray == f.getArray()
               && isFinal == f.getFinal()
               && type == f.getType();
    }
```

Type Analysis

- Works just like the ‘toString’ method
- Each node calls ‘typeCheck’ or ‘analyze’ on each child node.

- What should the return value be?
- How do we enforce our type checking?



Type Analysis

Start from the root node ***Program***

```
class Program extends Token {
 private RoutineList prog;
 //Constructor
 public Program(RoutineList r) {
   prog = r;
   symbolTable = new SymbolTable();
 }
……
 public String typeCheck() throws UTDLangException {
   return prog.typeCheck();
 }
}
```

Type Analysis

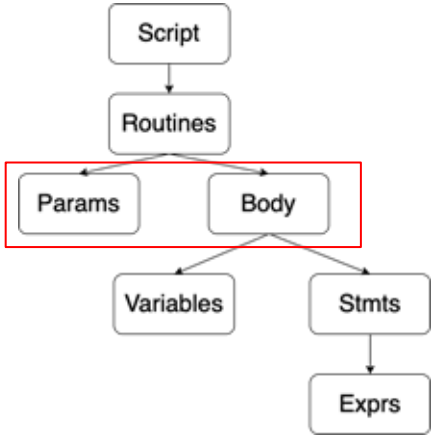
Going down to next level ***RoutineList***

```
public String typeCheck() throws UTDLangException {
  for (Routine r : routines) {
    String dummy = r.typeCheck();
  }
  return "";
}
```

Type Analysis

```
String id;
InOutList params;
Body body;
……
public String typeCheck() throws UTDLangException {
    symbolTable.addRoutine(id,params);
    symbolTable.startScope();
    String a = params.typeCheck();
    String b = body.typeCheck();
    symbolTable.endScope();
    return "";
 }
```

Routine.java



Type Analysis

Here we type check for each parameter by adding them into the new scope.

```
public String typeCheck() throws UTDLangException {
  symbolTable.addVar(id,type.toLowerCase());
  return "";
}
```

InParam.java

Type Analysis

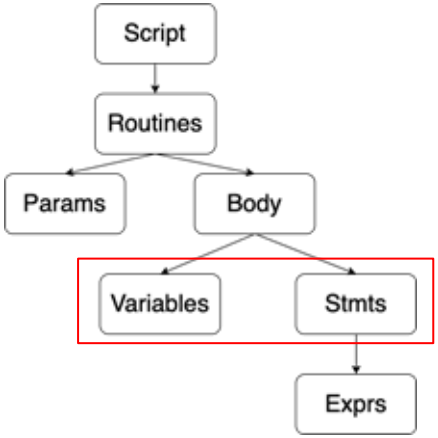
Then we type check for each variable or statement in a routine body.

```
public String typeCheck() throws UTDLangException {
  return stmtVar.typeCheck();
}
```

Body.java

```
public String typeCheck() throws UTDLangException {
  for (Statement s : stmtVar) {
    String dummy = s.typeCheck();
  }
  return "";
}
```

VarStmtList.java



Type Analysis

Here we type check for each variable by adding them into the current scope.

```
public String typeCheck() throws UTDLangException {
  symbolTable.addVar(id,type.toLowerCase());
  return "";
}
```

Variable.java

Type Analysis

Here we type check for Assignment Statements.

AssignmentStmt.java

```
String assignee;
Expr rhs;
……
public String typeCheck() throws UTDLangException {
  String type = symbolTable.get(assignee);
  String toAssign = rhs.typeCheck();
  if (!type.equals(toAssign)) {
    throw new UTDLangException("Error: tried to assign " + toAssign + " to type " + type
+ " : " + toString(0));
  }
  return "";
}
```

Type Analysis

Here we type check for Call Statements.

CallStmt.java

```
String callee;
IDList ids;
……
public String typeCheck() throws UTDLangException {
  String paramReqs = symbolTable.get(callee);
  String idTypes = ids.typeCheck();
  if (!paramReqs.equals(idTypes)) {
    throw new UTDLangException("Error: incompatible parameters" + " : " + toString(0));
  }
  return "";
}
```

Type Analysis

Here we type check for When Statements.

WhenStmt.java

```
Expr cond;
VarStmtList statements;
……
public String typeCheck() throws UTDLangException {
  String condType = cond.typeCheck();
  if (!condType.equals("flag")) {
    throw new UTDLangException("Error: tried to evaluate when condition which isn't a
flag" + " : when " + cond.toString(0) + " do");
  }
  statements.typeCheck();
  return "";
}
```

Type Analysis

Here we type check for Binary Expressions.

```
public String typeCheck() throws UTDLangException {
  String lhsType = lhs.typeCheck();
  String rhsType = rhs.typeCheck();
  if (!lhsType.equals(rhsType)) {
    throw new UTDLangException("Error: tried to operate on different types" + " : " +
toString(0));
  }
  if (lhsType.equals("number")) {
    if (operator.equals("?")) {
      return "flag";
    }
    return "number";
  }
```

BinExpr.java

Type Analysis

Here we type check for Binary Expressions.

BinExpr.java

```
 ……
else if (lhsType.equals("flag")) {
     if (operator.equals("+") || operator.equals("*")) {
       return "flag";}
     throw new UTDLangException("Error: tried to use " + operator + " on a flag " + " : " +
toString(0));}
   else { //String
     if (operator.equals("+")) {
       return "string";
     }
     throw new UTDLangException("Error: tried to use " + operator + " on a String " + " : " +
toString(0));
   }
```

When to throw an error?

- All rules are stated in the project description…. But in short:
  - Incompatible types for operation
    - ```
      if (!rhsType.equals("flag")) //flip only defined for flags
        throw new UTDLangException("Error: can't flip a " + rhsType + " : " + toString(0));
      ```
  - Redefinition of variable (Perfectly ok if in different scopes!!!)
    - ```
      for (Pair<String,String> p : table.get(table.size()-1)) //for all values in current
      scope  if (p.getKey().equals(id)) //if the id exists, it's redefined
        throw new UTDLangException("Error: tried to redeclare variable " + id);
      ```
  - Use of undeclared variable
    - ```
      for (int i = table.size()-1; i >= 0; --i) //for each scope level
        for (Pair<String,String> p : table.get(i))  //for each value in the table
          if (p.getKey().equals(s)) //if we find the id requested return the type
            return p.getValue();
      throw new UTDLangException("Error: variable not declared " + s); //id provided not in ANY scope
      ```
      ```

Things to Double Check

- Access of different scopes
  - ```
    Int x;

    Int main() {
          Int y = x; //this refers to the x in global scope
          Int x = 2;
          Int z = x; //this refers to the x defined in the local scope!
    }
    ```
- Make sure parameter types are correct!
- Make sure final variables ARE final!
- Make sure returned values of methods are correct
    ```